EXPERIMENT 10

Explore and Implement Autoencoders for Image Denoising

1.Dataset Source

Dataset Name: MNIST Dataset
Source: Kaggle
Link:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

The dataset contains grayscale images of handwritten digits used widely for image processing and deep learning tasks.

2.Dataset Description

The **MNIST dataset** consists of handwritten digit images (0–9).

Dataset Characteristics

- Total Images: 70,000
  - Training: 60,000
  - Testing: 10,000
- Image Size: 28 × 28 pixels
- Color: Grayscale (0–255 intensity values)

Features

Each image is represented as a matrix of pixel values:

$X \in R$28×28X∈R28×28

- Each pixel value ranges from **0 (black) to 255 (white)**

Target Variable

Variable	Description
Label	Digit (0–9)

Note:
For **Autoencoder**, labels are **not used**, since it is an **unsupervised learning algorithm**.

Dataset Characteristics

- Balanced dataset
- Low-resolution images
- Suitable for deep learning experiments
- Common benchmark dataset

3. Mathematical Formulation of the Algorithm

Autoencoder

An Autoencoder is a neural network used for **unsupervised learning**.

It consists of:

- **Encoder** → compresses input
- **Latent Space** → compressed representation
- **Decoder** → reconstructs input

Encoder Function

$z=f(x)=\sigma(Wx+b)$z=f(x)=σ(Wx+b)

Where:

- xx = input image
- zz = latent representation
- W,bW,b = weights and bias

Decoder Function

x^=$g(z)=\sigma(W'z+b')$x^=g(z)=σ(W′z+b′)

Where:

- x^x^ = reconstructed image

Loss Function (Reconstruction Loss)

Most commonly used:

Mean Squared Error (MSE)

L=1n∑(x−x^)2L=n1∑(x−x^)2

Denoising Autoencoder

Instead of clean input:

$xnoisy$=x+$noise$xnoisy=x+noise

Model learns:

f($xnoisy$)→xf(xnoisy)→x
Goal: Remove noise and reconstruct original image.

4.Algorithm Limitations

Autoencoder Limitations

- Requires large data for good performance
- May simply memorize input (overfitting)
- Cannot generate completely new data (basic autoencoder)
- Sensitive to noise level selection
- Poor performance on very complex images

Denoising Limitations

- Too much noise → reconstruction fails
- Too little noise → model learns identity mapping
- Blurred outputs may occur

5.Methodology / Workflow

Step 1: Data Loading

- Load MNIST dataset
- Normalize pixel values (0–1)

Step 2: Add Noise

- Gaussian noise added to images:

$xnoisy$=x+N(0,σ2)xnoisy=x+N(0,σ2)

Step 3: Model Architecture

- Encoder:
  - Dense / Conv layers
- Latent Space:
  - Compressed feature representation
- Decoder:
  - Reconstruct original image

Step 4: Model Training

- Input: Noisy images
- Output: Clean images
- Loss: MSE
- Optimizer: Adam

Step 5: Prediction

- Feed noisy images
- Generate denoised output

Step 6: Visualization

- Original Image
- Noisy Image
- Reconstructed Image

Workflow Diagram

Clean Image

↓

Add Noise

↓

Noisy Image

↓

Encoder

↓

Latent Space

↓

Decoder

↓

Reconstructed Image

6. Performance Analysis

Evaluation Metrics

1. Mean Squared Error (MSE)

- Measures reconstruction error
- Lower value = better performance

2. Visual Comparison (Important)

Compare:

- Original Image
- Noisy Image
- Reconstructed Image

Key Observations:

- Noise reduced successfully
- Image structure preserved
- Slight blurring may occur

Results Interpretation

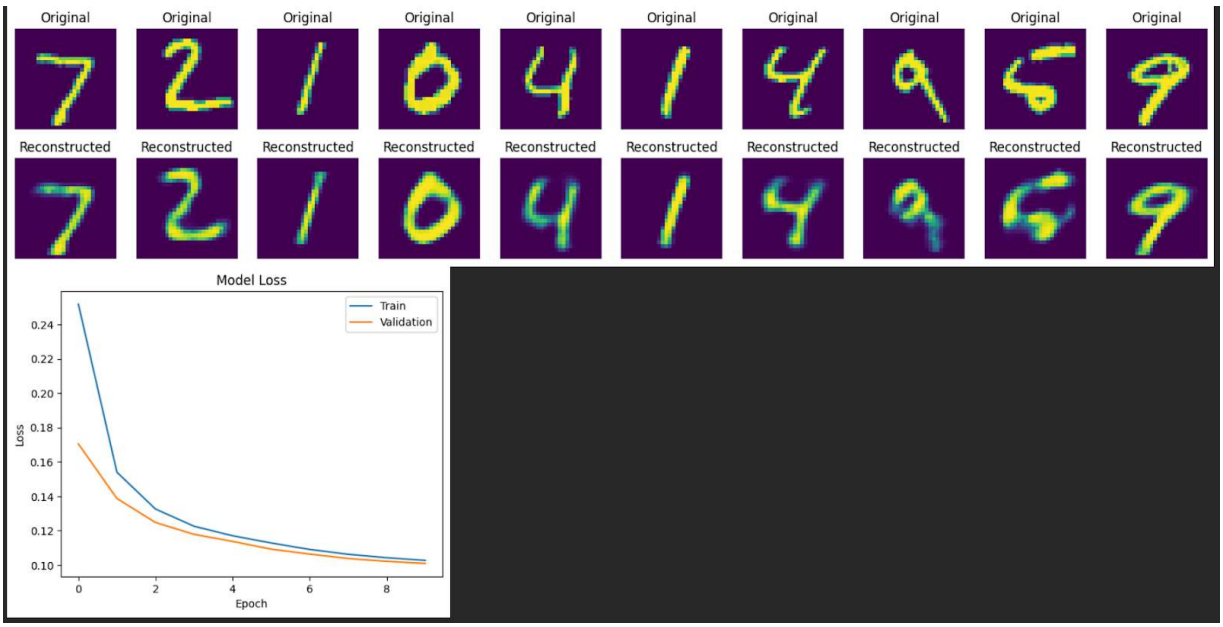
- Good reconstruction → model learned meaningful features
- High error → model underfitting
- Perfect reconstruction → possible overfitting

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datas
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training shape: (60000, 784)
Testing shape: (10000, 784)
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 64)	2,112
dense_4 (Dense)	(None, 128)	8,320
dense_5 (Dense)	(None, 784)	101,136

```
 Total params: 222,384 (868.69 KB)
 Trainable params: 222,384 (868.69 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - loss: 0.2518 - val_loss: 0.1705
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - loss: 0.1540 - val_loss: 0.1387
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - loss: 0.1326 - val_loss: 0.1247
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - loss: 0.1226 - val_loss: 0.1178
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 21ms/step - loss: 0.1170 - val_loss: 0.1137
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - loss: 0.1128 - val_loss: 0.1092
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - loss: 0.1090 - val_loss: 0.1064
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 19ms/step - loss: 0.1062 - val_loss: 0.1038
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - loss: 0.1042 - val_loss: 0.1021
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - loss: 0.1027 - val_loss: 0.1009
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

- 

7. Hyperparameter Tuning

Tuned Parameters

Parameter	Description
Latent dimension	Size of compressed layer
Learning rate	Controls training speed
Batch size	Number of samples per iteration
Epochs	Training cycles
Noise factor	Amount of noise added

Example Tuning

- Latent size: 32, 64, 128
- Learning rate: 0.001, 0.0001
- Noise factor: 0.2, 0.4, 0.6

Impact of Tuning

- Larger latent space → better reconstruction
- Smaller latent space → more compression
- Optimal noise → best denoising performance

Conclusion

This experiment demonstrated:

- Implementation of Autoencoder for unsupervised learning
- Image denoising using neural networks
- Feature compression and reconstruction
- Effect of noise on model learning

Autoencoders are powerful tools for:

- Image denoising
- Dimensionality reduction
- Feature extraction